

System Call Investigation

Taaruni Ananya | 002757903 | CSC 4320 | September 16

AI Response:

Great, let's walk through this like we're reading a story about a program's life. Every line in these traces is a **system call** — a moment where your program stops doing its own thing and asks the kernel (the OS's core) to do something on its behalf, like opening a file or printing text.

Trace A — the successful run

A03: `openat(AT_FDCWD, "test.txt", O_RDONLY) = 3`

The program asks the kernel: "please open test.txt, read-only." The kernel finds the file, sets up some internal bookkeeping for it, and hands back a small number, **3**. That number is a **file descriptor** — think of it like a claim ticket. The program doesn't get the file itself; it gets a ticket number it can use in future calls to refer to "that open file."

A04: `read(3, "Hello, OS Kernel!\n", 100) = 18`

The program says: "kernel, using ticket 3, read up to 100 bytes for me." The kernel copies data from the file into the program's memory. The return value, **18**, is *not* an error code here — it's the **number of bytes actually read**. The file only had 18 bytes ("Hello, OS Kernel!" plus a newline), so even though the program was willing to accept up to 100, it only got 18.

A05: `write(1, "Hello, OS Kernel!\n", 18) = 18`

Now the program says: "kernel, using ticket 1, write these 18 bytes out." Ticket 1 is standard output (your terminal screen). The return value **18** confirms all 18 bytes were successfully written. Again, this return value is a count of bytes, not a status flag.

A06: `close(3) = 0`

The program is done with the file, so it tells the kernel to release ticket 3 — the file descriptor becomes invalid and the kernel can clean up its bookkeeping. The return value **0** here just means "success" (unlike read/write, close doesn't return a byte count — it returns 0 for success or -1 for failure).

Trace B — the failed run

B01: `openat(AT_FDCWD, "test.txt", O_RDONLY) = -1 ENOENT`

The program tries the same open call. But this time the file doesn't exist, so the kernel can't give out a ticket. It returns **-1**, which is the universal "this call failed" signal, and attaches an error code, **ENOENT**. That stands for "**Error, NO ENTry**" — meaning there's no directory entry (no file) with that name. It's the kernel's specific way of saying "I looked, and that file just isn't there."

B02: `write(2, "open: No such file or directory\n", 32) = 32`

The program reacts to the failure by printing an error message. Notice it writes to file descriptor **2**, not **1**. This is standard **error output** — a separate channel from normal output, specifically for error/diagnostic messages. All 32 bytes were written successfully (hence the

return value 32).

B03: `exit_group(1) = ?`

The program tells the kernel it's finished, and it's exiting with status code `1`. By convention, an exit code of `0` means success and any nonzero value (like `1`) signals that something went wrong. The `= ?` just reflects that exit calls don't return in the normal sense — the process is gone, so there's nothing to return to.

B04: `+++ exited with 1 +++`

This is just the tracing tool (like `strace`) noting, from the outside, that the process has fully terminated with exit status `1`.

File descriptors 1, 2, and 3

Every process starts with three file descriptors already open automatically, before it does anything:

- `0` = standard input (keyboard/input stream) — not used in either trace here
- `1` = standard output — normal program output, like the greeting printed in A05
- `2` = standard error — error messages, like B02

Descriptor `3` is different: it's not automatic. It was handed out dynamically by the `openat` call in A03, because `0`, `1`, and `2` were already taken. If the program opened a second file, it would typically get descriptor `4`, and so on.

Kernel mode vs. switching to another process

These are two different concepts that are easy to mix up:

Entering kernel mode for a system call is about *privilege level*, not about *which program runs*. Your program (`./read_file`) can't directly touch the disk or the screen itself — hardware access is protected. So when it calls `open`, `read`, `write`, etc., the CPU switches from "user mode" (restricted) to "kernel mode" (privileged), the kernel does the actual work (talking to the disk, managing memory), and then control switches back to user mode and execution resumes in the *same* program, right after the system call. No other process gets to run in between, necessarily — it's the same process, just temporarily letting the kernel act on its behalf.

Switching to another process (called a **context switch**) is a completely different event. This happens when the kernel's scheduler decides your program's time slice is up, or your program is blocked waiting on something slow (like disk I/O), and the CPU should go run some *other* process for a while. The kernel saves all of your process's state (registers, program counter, etc.) so it can resume later, then loads another process's saved state and lets it run.

The key distinction: a system call is your *own* process briefly handing control to the kernel and getting it back; a context switch is the CPU leaving your process entirely to go run a *different* process. A system call can sometimes trigger a context switch (e.g., if `read` has to wait for a

slow disk and the kernel decides to run someone else while waiting), but they're conceptually separate mechanisms — one is about *privilege*, the other is about *scheduling*.

Question 1: Successful execution (25 points)

A03 requests the OS to open test.txt as a read only file, which prompts the kernel to open it and return the descriptor 3. A04 now reads descriptor 3 and 18 bytes are read. A05, the 18 bytes are written to descriptor 1 in the write() statement to the screen user. For A06, descriptor 3 is closed and close() succeeded because it has a return value of 0, indicating a success.

Question 2: Failed execution (20 points)

openat() fails in B01 because test.txt does not exist, indicated by the failed -1 return value. The -1 value means the call failed while the ENOENT value means that there does not exist a directory entry for the given name in the statement; it stands for “Error, no entry.” The error message is written to descriptor 2 in B02 because of the separate channel that exists only to display messages when there are issues, rather than to the normal output channel. There are no successful read() or close() calls for test.txt after B01 because there isn’t a test.txt file to read or close at all.

Question 3: Program and trace connection (20 points)

Program line	Operation	Trace line to identify
Line 7	Open test.txt	_____
Line 11	Read from test.txt	_____
Line 16	Write the file contents	_____
Line 25	Close test.txt	_____

Line 7’s open test.txt can be traced A03 because while the C code is requesting with open(), the OS kernel runs it as openat() and matches the request to read test.txt. Line 11’s read from test.txt can be tracked to A04 where the C code requests read() and the kernel requests to read test.txt, with the descriptor 3 that was assigned in A03. Line 16’s write the file contents matches A05 request to write() the 18 bytes

where `write(1, "Hello, OS Kernel!\n", 18) = 18`. Line 25's `close test.txt` matches A06's `close()` call that closes descriptor 3, which was assigned to `test.txt`.

Question 4: Kernel mode and context switch (20 points)

A03, A04, A05, and A06 operations in Trace A require help from the kernel because they need services that only the kernel can provide such as `openat()`, `read()`, `write()`, and `close()`. Line 7's `int fd = open("test.txt", O_RDONLY);` can be traced to A03 `openat(AT_FDCWD, "test.txt", O_RDONLY) = 3` and Line 25's `if (close(fd) == -1) { perror("close"); return 1; }` can be traced to A06 `close(3) = 0`. A program must enter kernel mode to open or read a file because the kernel is the only one that can access the file from memory to read and open. Entering kernel mode does not necessarily mean that the CPU starts running a different process because the kernel is the one handling the current process's request and returning it once complete, leaving no need for the CPU to start running another process. Strace can show a program's system call, arguments, and any returned data. It can't directly show the CPU's mode bit as it switches between user and kernel mode; it also can't directly show if a context switch occurred or not to another process.

Question 5: Evaluate the AI Response (15 points)

Chosen AI statement segment:

Trace A — the successful run

A03: `openat(AT_FDCWD, "test.txt", O_RDONLY) = 3`

The program asks the kernel: "please open test.txt, read-only." The kernel finds the file, sets up some internal bookkeeping for it, and hands back a small number, **3**. That number is a **file descriptor** — think of it like a claim ticket. The program doesn't get the file itself; it gets a ticket number it can use in future calls to refer to "that open file."

A04: `read(3, "Hello, OS Kernel!\n", 100) = 18`

The program says: "kernel, using ticket 3, read up to 100 bytes for me." The kernel copies data from the file into the program's memory. The return value, **18**, is *not* an error code here — it's the **number of bytes actually read**. The file only had 18 bytes ("Hello, OS Kernel!" plus a newline), so even though the program was willing to accept up to 100, it only got 18.

A05: `write(1, "Hello, OS Kernel!\n", 18) = 18`

Now the program says: "kernel, using ticket 1, write these 18 bytes out." Ticket 1 is standard output (your terminal screen). The return value **18** confirms all 18 bytes were successfully written. Again, this return value is a count of bytes, not a status flag.

A06: `close(3) = 0`

The program is done with the file, so it tells the kernel to release ticket 3 — the file descriptor becomes invalid and the kernel can clean up its bookkeeping. The return value **0** here just means "success" (unlike read/write, close doesn't return a byte count — it returns 0 for success or -1 for failure).

I verified the statements by directly tracing back its responses to the Trace and each identifier's request. For A03, the AI response correctly says that the program requests the kernel to open test.txt as a read only with a file descriptor of 3, this can be seen in A03 where "**A03 `openat(AT_FDCWD, \"test.txt\", O_RDONLY) = 3`**" prove the AI's response that test.txt is requested to be opened with a file descriptor of 3. I used the same verification method of tracing each AI response for each A0-line and bolded each statement that connects to the response:

A04: A04 **`read(3, \"Hello, OS Kernel!\\n\", 100) = 18`**, the program reads 18 bytes from the file descriptor 3, as the AI response also correctly states.

A05: A05 **`write(1, \"Hello, OS Kernel!\\n\", 18) = 18`**, the program writes 18 bytes to file descriptor 1 the screen, as the AI response also correctly states.

A06: A06 **`close(3) = 0`**, the program closes file descriptor 3 successfully (as indicated by the 0 return value), as the AI response also correctly states.